

The Knife

Manuale tecnico

Progetto Laboratorio
Interdisciplinare A

The Knife

Versione

1.4

Autori

Tommaso Ossuzio,
Elia Toschi, Matteo Franguelli,
Celestino Resteghini

Indice

Introduzione _____	3
Struttura generale dell'applicazione _____	3
Scelte architettureali _____	3
Strutture dati _____	5
Algoritmi utilizzati _____	6
Limiti della soluzione sviluppata _____	7
Sitografia / Bibliografia _____	8

Introduzione

TheKnife è una piattaforma che consente di trovare ristoranti in tutto il mondo e selezionarli in base al luogo, alla tipologia del ristorante stesso, alla fascia di prezzo, alla possibilità o meno di prenotare un tavolo o di ordinare da asporto. TheKnife, dunque, simula alcune delle funzionalità della celebre piattaforma TheFork

Struttura generale dell'applicazione

Scelte architetturali

L'applicazione è strutturata secondo una separazione chiara delle responsabilità (ispirata al pattern MVC e adattata a JavaFX), distinguendo tra:

- *modello dei dati e della logica applicativa*
- *gestione dell'interfaccia grafica*
- *gestione dello stato globale dell'applicazione*

Questa scelta migliora manutenibilità, leggibilità e riusabilità del codice, riducendo l'accoppiamento tra le varie parti del sistema.

Struttura dei package

Package theknife.model -

Contiene le classi che rappresentano il dominio applicativo e la logica di gestione dei dati.

Classi principali:

➤ **Ristorante**

Rappresenta un ristorante, con le informazioni principali (nome, coordinate, sito web, ecc..)

- **Recensione**
Modella una recensione lasciata da un utente, includendo testo e valutazione in stelle
- **Risposta**
Rappresenta la risposta di un ristoratore a una recensione.
- **Luogo**
Incapsula le informazioni geografiche (es. coordinate), usate per il filtraggio e la visualizzazione.
- **GestioneRistoranti**
Classe di servizio che si occupa delle operazioni sui ristoranti
- **GestioneRecensioni**
Gestisce la logica legata alle recensioni
- **GestioneFile**
Centralizza la persistenza dei dati su file, separando la logica di I/O dal resto dell'applicazione.

Questo package non dipende dall'interfaccia grafica, rispettando il principio di separazione dei livelli.

Package **theknife.ui.javaafx** -

Contiene le classi dedicate all'interfaccia utente JavaFX e alla gestione delle interazioni.

Classi principali:

- **MainApp**
Classe di avvio dell'applicazione. Inizializza lo stage principale e carica le viste.
- **MainController**
Controller centrale che gestisce la navigazione tra le schermate principali.
- **Controller specifici per funzionalità**
Ogni vista ha un controller dedicato, ad esempio:
 - **LoginController** → gestione login
 - **RegisterController** → registrazione utente
 - **AddRestaurantController** → inserimento ristorante
 - **AddReviewController** → inserimento recensioni
 - **FavoritesController** → gestione dei ristoranti preferiti
 - **RestaurantDetailsController** → dettagli di un ristorante
 - **ViewReviewsController / MyReviewsController** → visualizzazione recensioni (rispettivamente per Cliente e Ristoratore)
 - **ReplyReviewsController** → risposte alle recensioni
 - **AdvancedFilterController** → filtro avanzato

La gestione dell'utente autenticato è affidata alla classe Session, che rappresenta la sessione corrente dell'applicazione.

La classe è implementata secondo il **Singleton Pattern**, garantendo l'esistenza di un'unica istanza condivisa per tutta l'esecuzione del programma.

Session mantiene le informazioni relative all'utente loggato, al ruolo principale e ai permessi associati, consentendo alle varie componenti dell'interfaccia grafica di adattare il proprio comportamento in base allo stato della sessione.

Strutture dati

Per la gestione e la memorizzazione dei dati, l'applicazione utilizza file in formato CSV (Comma-Separated Values) come principale struttura dati persistente.

La scelta del formato CSV è motivata dalla sua semplicità, leggibilità e facilità di utilizzo all'interno di un progetto di dimensioni contenute. Il file CSV consente di rappresentare i dati in forma tabellare, dove ogni riga corrisponde a un elemento (ad esempio un ristorante o una recensione) e ogni colonna rappresenta un attributo specifico.

Dal punto di vista implementativo, il CSV permette:

- una lettura e scrittura semplice tramite le librerie standard di Java;
- un accesso diretto ai dati senza la necessità di configurare un database;
- una facile modifica manuale dei dati per scopi di test e debugging.

I dati letti dal file CSV vengono caricati in memoria all'avvio dell'applicazione e trasformati in strutture dati Java (come liste di oggetti), permettendo una gestione efficiente delle operazioni di ricerca e filtraggio durante l'esecuzione.

Questa soluzione è risultata adeguata agli obiettivi del progetto, in quanto consente di mantenere il sistema leggero, portabile e di semplice manutenzione. Tuttavia, l'utilizzo di file CSV comporta alcune limitazioni in termini di scalabilità e gestione concorrente dei dati, aspetti che potrebbero essere migliorati in futuri sviluppi tramite l'adozione di un database.

Nel progetto è stata utilizzata una **ObservableList** per la gestione dei dati relativi ai ristoranti importati dal file CSV.

L'ObservableList è una struttura dati fornita da Java che permette di monitorare automaticamente le modifiche apportate alla lista, come l'aggiunta, la rimozione o l'aggiornamento degli elementi.

Questa scelta è stata effettuata per facilitare l'integrazione tra il modello dei dati e la parte grafica dell'applicazione. In particolare, l'uso di una ObservableList consente di mantenere la vista grafica sincronizzata con i dati: ogni modifica alla lista dei ristoranti viene immediatamente riflessa nell'interfaccia utente, senza la necessità di aggiornamenti manuali.

Grazie a questo meccanismo, i dati provenienti dal file CSV possono essere visualizzati in modo dinamico e coerente all'interno dell'applicazione, migliorando l'organizzazione del codice e la gestione dell'interazione tra dati e grafica.

Algoritmi utilizzati

– Algoritmo di ricerca:

Radix Sort (MSD) è un algoritmo di ordinamento non comparativo: a differenza di algoritmi come QuickSort o MergeSort, non confronta direttamente gli elementi tra loro, ma analizza i caratteri (o le cifre) che compongono i dati da ordinare. Nel contesto del presente software, l'algoritmo opera esclusivamente su stringhe di caratteri.

Nella variante MSD (Most Significant Digit), l'ordinamento avviene a partire dal carattere più significativo, ovvero il primo carattere da sinistra.

L'algoritmo esamina tutte le stringhe e le distribuisce in strutture ausiliarie denominate bucket (o liste temporanee), ciascuna associata a uno specifico valore del carattere in esame.

Una volta completata la distribuzione nei bucket, l'algoritmo procede in modo ricorsivo: per ogni bucket non vuoto, viene ripetuto lo stesso processo sul carattere successivo. In questa fase, vengono considerati esclusivamente gli elementi appartenenti al bucket corrente, mentre gli altri vengono temporaneamente esclusi dall'elaborazione.

È importante sottolineare che il Radix Sort non elimina mai dati: la selezione dei sottoinsiemi avviene unicamente a livello logico, per consentire l'ordinamento progressivo delle stringhe in base ai caratteri successivi. Al termine della ricorsione, la concatenazione ordinata dei bucket produce la sequenza finale correttamente ordinata.

- *Algoritmo di Haversine per calcolo delle distanze:*

L'algoritmo di Haversine è utilizzato per calcolare la distanza tra due punti sulla superficie terrestre, assumendo la Terra come una sfera. Il calcolo si basa sulle coordinate geografiche (latitudine e longitudine) espresse in gradi.

Nel software, l'algoritmo viene applicato utilizzando le coordinate contenute nella classe *Luogo*. A partire da tali coordinate, viene calcolata la distanza tra la posizione di riferimento e la posizione del ristorante. Il risultato del calcolo viene quindi confrontato con una soglia prefissata, pari a 10 km, al fine di verificare se il ristorante rientra nel raggio definito dall'utente.

Limiti della soluzione sviluppata

- *Gestione utenti limitata*

L'applicazione non è concorrenziale: non è prevista la gestione di più utenti contemporaneamente.

- *Autenticazione semplificata*

Il sistema di accesso non prevede meccanismi avanzati di sicurezza (es. crittografia delle credenziali, gestione dei ruoli, sessioni persistenti).

- *Scalabilità ridotta*

L'architettura non è progettata per supportare un elevato numero di ristoranti, recensioni o utenti.

- *Persistenza dei dati limitata*

I dati possono essere memorizzati localmente o in modo non persistente, senza l'utilizzo di un database distribuito o remoto.

- *Assenza di sincronizzazione in tempo reale*

Le modifiche ai dati non vengono propagate in tempo reale tra più istanze dell'applicazione.

- *Funzionalità avanzate non implementate*

Non sono presenti sistemi di raccomandazione, rating aggregato o analisi delle recensioni.

- *Interfaccia utente essenziale*

L'interfaccia è funzionale ma non ottimizzata per accessibilità, personalizzazione o supporto multi-lingua.

Sitografia / Bibliografia

- <https://app.diagrams.net>
- <https://docs.oracle.com/en/java/javase/21/docs/api/index.html>
- <https://material.io>
- <https://www.opentable.com>
- <https://www.thefork.it>
- <https://www.jetbrains.com/help/idea/javadoc.html>
- <https://openjfx.io>
- <https://mvnrepository.com/artifact/org.openjfx/javafx-controls>

Il manuale utente è stato redatto da:

~ Tommaso Ossuzio